

DOCUMENTO DE REQUERIMIENTOS TÉCNICOS (TRD)

Arquitectura ResqueNet: Sostenibilidad Open Source y Despliegue Crítico Solitario

Autor de la Base	Arquitecto de Software Senior / Lead Developer	Estrategia de Código	Open Source (AGPL-3.0-or-later)
Stack Principal	Django 5.x / Vue 3 / PostGIS / Redis	Estado Tecnológico	Listo para Desarrollo

ENFOQUE DE SOSTENIBILIDAD (SOLO-DEVELOPER A COMUNIDAD): Este documento detalla las directrices de ingeniería necesarias para que un único desarrollador pueda levantar, desplegar y mantener el núcleo de la plataforma de manera eficiente, estableciendo a su vez una arquitectura modular y herramientas automáticas de calidad de código para permitir la incorporación orgánica de contribuidores de la comunidad Open Source.

1. Arquitectura de Software y Principios SOLID

Para evitar que el código se convierta en una estructura rígida e inmantenible, el backend se diseña bajo los principios de **Clean Architecture**, dividiendo el sistema en capas con dependencias estrictas hacia el interior:

- **Capa de Dominio (Domain):** Contiene las entidades puras de negocio (ej. `AffectedPerson`) y reglas lógicas independientes de frameworks. Aplica el principio de Responsabilidad Única (SRP).
- **Capa de Casos de Uso (Application / Use Cases):** Orquesta el flujo de datos desde y hacia las entidades. Implementa interfaces de repositorios para cumplir con el principio de Inversión de Dependencias (DIP).
- **Capa de Infraestructura (Infrastructure):** Implementación de detalles tecnológicos (Django ORM, PostGIS, drivers de Redis, pasarelas de SMS). Al aislar la infraestructura, un cambio en la base de datos no rompe la lógica de negocio.
- **Capa de Interfaces (Interfaces / Presentation):** Controladores de la API REST, serializadores y consumidores de WebSockets (Django Channels).

2. Estructura del Entorno de Contribución Abierta

Para automatizar la validación de código proveniente de terceros y evitar revisiones manuales extenuantes para el mantenedor principal, se implementa una suite estricta de validación local mediante hooks de Git:

```
# Archivo: .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.4.0
    hooks:
      - id: check-yaml
      - id: end-of-file-fixer
      - id: trailing-whitespace
  - repo: https://github.com/psf/black
    rev: 23.3.0
```

```

hooks:
  - id: black # Formateador automático estricto de Python
- repo: https://github.com/pycqa/flake8
  rev: 6.0.0
hooks:
  - id: flake8 # Linter para asegurar cumplimiento de PEP 8
    args: [--max-line-length=100]

```

3. Diseño Detallado del Modelo de Datos (Capa de Infraestructura Django ORM)

A continuación se detalla la implementación en código del modelo geoespacial relacional utilizando el motor PostGIS, garantizando indexación rápida para escenarios como el deslave de Portuguesa o inundaciones generalizadas:

```

from django.contrib.gis.db import models
import uuid

class AffectedPerson(models.Model):
    STATUS_CHOICES = [
        ('MISSING', 'Desaparecido'),
        ('FOUND', 'Encontrado Vivo'),
        ('RESCUED', 'Rescatado'),
        ('EVACUATED', 'Evacuado en Refugio'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    first_name = models.CharField(max_length=100, db_index=True)
    last_name = models.CharField(max_length=100, db_index=True)
    document_id = models.CharField(max_length=50, blank=True, null=True, db_index=True)
    age = models.IntegerField(blank=True, null=True)
    current_status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='MISSING')
    medical_conditions = models.TextField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

class LocationReport(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    person = models.ForeignKey(AffectedPerson, on_delete=models.CASCADE,
related_name='locations')
    geom = models.PointField(srid=4326) # Campo geométrico espacial (Longitud, Latitud)
    address_description = models.TextField()
    is_verified = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    indexes = [
        models.Index(fields=['created_at']),
    ]

```

4. Especificación de Endpoints y Contrato de API REST

Consulta de Proximidad Geoespacial

Endpoint: GET /api/v1/locations/nearby/

Permite a los rescatistas mapear en tiempo real los incidentes en un radio determinado mediante consultas nativas de PostGIS (`ST_DWithin`).

```
# Ejemplo de respuesta exitosa (200 OK)
[
  {
    "person_id": "8f3c8346-608a-4d2d-9610-84cf05bfd419",
    "full_name": "María Mendoza",
    "status": "MISSING",
    "coordinates": {
      "latitude": 9.3456,
      "longitude": -69.2134
    },
    "distance_meters": 450.2
  }
]
```

5. Arquitectura Asíncrona, Tiempo Real y Gestión de Colas

Para evitar cuellos de botella cuando miles de ciudadanos refrescan el sitio web simultáneamente, la persistencia y la notificación de eventos se separan usando un patrón de arquitectura dirigida por eventos (EDA):

1. **Ingesta de Eventos:** Las solicitudes de actualización críticas entran por la API REST y se guardan de forma transaccional rápida.
2. **Broker de Mensajería:** Django publica un evento estructurado en un canal de **Redis Pub/Sub**.
3. **Distribución Asíncrona:** Los servidores asíncronos de ****Django Channels**** (ASGI corriendo bajo Daphne/ Uvicorn) consumen de Redis y mantienen conexiones persistentes via WebSockets con el cliente Vue.js de forma no bloqueante.
4. **Procesamiento Pesado:** El procesamiento de archivos multimedia (compresión de imágenes de evidencias) se encola inmediatamente en ****Celery****, liberando el hilo HTTP en menos de 50ms.

6. Estrategia Offline-First (Resiliencia ante Caídas de Conectividad)

Dado el contexto de vulnerabilidad de la infraestructura de telecomunicaciones en situaciones de desastre, el frontend implementa un mecanismo de persistencia local transparente:

- **Service Worker (Workbox):** Almacena en caché los assets estáticos del frontend (HTML, JS, CSS, mapas base de OpenStreetMap en formato vectorial offline). La aplicación cargará aun si el dispositivo está en modo avión.

- **IndexedDB Capa de Datos:** Cuando un usuario genera un reporte sin señal de red, el composable de Vue intercepta el error de red, escribe el payload en la base de datos local del navegador (IndexedDB) y añade el registro a una cola de sincronización.
- **Background Sync API:** Tan pronto como el navegador detecta el evento `online`, se activa un subproceso en segundo plano que vacía la cola local enviando peticiones en lote (*batch processing*) a la API central, resolviendo conflictos priorizando la marca de tiempo (Timestamp) más reciente.

7. Despliegue Automatizado y DevOps Simplificado (Solo Dev Focus)

Para que el proyecto sea administrable por una sola persona sin consumir horas en tareas operativas, se utiliza un pipeline de integración y despliegue continuo (CI/CD) basado en GitOps:

Componente	Tecnología Seleccionada	Razón Solitario / Open Source
Entorno Local	Docker Compose (App + DB + Redis)	Permite que un nuevo colaborador corra el proyecto con un solo comando: <code>docker-compose up</code> .
Pipeline CI/CD	GitHub Actions (Pruebas + Build de imágenes)	Gratuito para proyectos Open Source, ejecuta linters y pruebas automáticas en cada Pull Request.
Orquestación	K3s / Kubernetes Liviano o Docker Swarm	Bajo consumo de memoria ram, ideal para servidores VPS económicos pero con soporte de autorecuperación.
Monitoreo	GlitchTip / Sentry (Open Source) + Prometheus	Notifica errores críticos directamente al correo o canal de comunicación del administrador en tiempo real.

8. Estrategia de Pruebas de Carga de Emergencia

Para asegurar que el sistema no colapse ante picos masivos de tráfico unificados por un desastre, se define una prueba de estrés automatizada obligatoria usando **Locust**. El script simula el comportamiento masivo de ciudadanos realizando búsquedas concurrentes de familiares:

```
from locust import HttpUser, task, between

class EmergencyTrafficSimulation(HttpUser):
    wait_time = between(0.5, 2) # Simula usuarios reales refrescando constantemente

    @task(3)
    def search_by_name(self):
        # Simula consulta al buscador de víctimas (Fase crítica)
```

```
self.client.get("/api/v1/people/search/?name=Alex")

@task(1)
def send_location_ping(self):
    # Simula a un rescatista enviando coordenadas GPS consecutivas
    self.client.post("/api/v1/people/report/", json={
        "person_id": "8f3c8346-608a-4d2d-9610-84cf05bfd419",
        "current_status": "EVACUATED",
        "location": {"latitude": 10.2, "longitude": -67.5}
    })
```